



University of
Nottingham

UK | CHINA | MALAYSIA

A large, high-resolution image of the Earth as seen from space, showing the Western Hemisphere with North and South America. The Earth is framed by a thin white rectangular border. The background is a deep black space filled with numerous small, distant stars.

COMP-2024 / COMP-2039
**Artificial Intelligence
Methods**

Lecture 6
Evolutionary Algorithm II



Learning Outcomes

IDENTIFY

1. Travelling Salesman Problem
2. Generic Permutation based Genetic Operators
3. Multimeme Memetic Algorithms
4. Benchmark Function Optimisation





University of
Nottingham

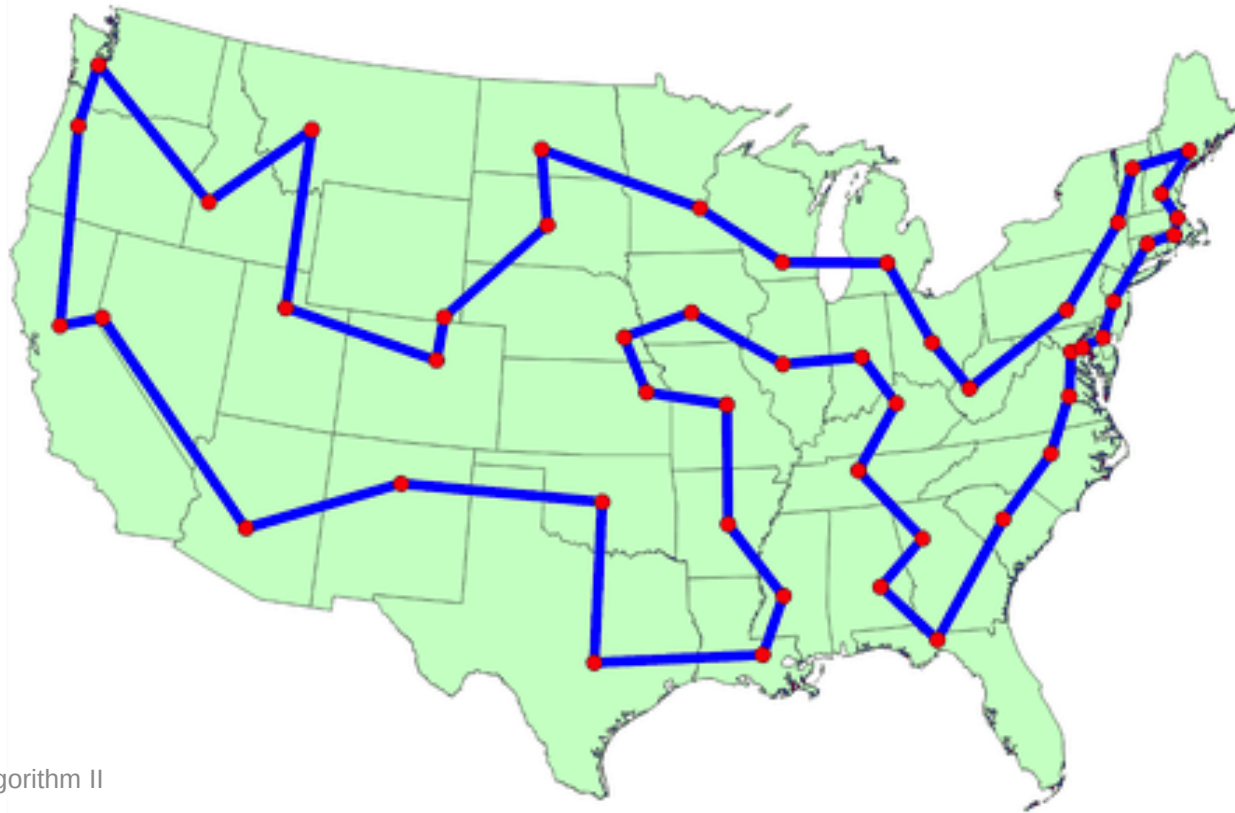
UK | CHINA | MALAYSIA

Travelling Salesman Problem in Genetic Algorithm



Traveling Salesman Problem (TSP)

- In the TSP, a salesman needs to visit a set of cities exactly once and return to the original city, minimizing the total travel distance.
- Each unique order in which the cities are visited can be represented as a permutation.





Can binary representation be used for **TSP**
in Genetic Algorithm as in MAX-SAT?



Binary Representation for Encoding Permutation in TSP – The Issues

- Binary representation (e.g. 1000111) and classical random initialisation, cross-over and mutation operators are not suitable for encoding **permutation**.
 - E.g., for TSP, could cause illegal tours to form.
 - not all cities visited
 - undefined city codes
 - cities visited more than once
 - loops formed within the tour
 - needs *repair algorithms* - to fix invalid individuals, adding complexity and computational overhead.
 - Binary encoding and traditional crossover/mutation operators are **unsuitable for TSP** as they *violate the permutation property*.



Problems with Binary Encoding

- Some sequences may result in invalid tours:
 - **Not all cities are visited** (e.g., city codes missing).
 - **Undefined city codes** may arise when the binary sequence represents non-existent cities (e.g., for $N = 5$, 6, 7 are invalid).
 - **Duplicate cities** in the tour (a city is visited more than once).
 - **Loops** formed within the tour.



Example (The Issues)

Given $N=5$ cities $\Rightarrow$ 3 bits per city ($2^3=8>5$). E.g.: City 2 (010), City 5 (101).
An individual is a sequence of binary strings, representing a tour.

	2	5	3	1	4		2		5	3	1	4		
individual 1:	0		1	0	1	0	1	1	0	0	1	1	0	0

	5	2	1	4	3		5		2	1	4	3		
individual 2:	1		0	1	0	0	0	1	1	0	0	0	1	1

After applying 1-point crossover (after first bit):

	1	2	1	4	3		2	2	1	4	3			
individual 1:	0	0	1	0	1	0	0	1	1	0	0	0	1	1
	6	5	3	1	2		5	5	3	1	4			
individual 2:	1	1	0	1	0	1	0	1	1	0	0	1	1	0



1-Point Crossover After the First Bit

Step 1: Understanding the Parent Individuals

The two parent individuals are encoded as binary strings, but they map to city sequences:

Before Crossover

- Individual 1: 010101011001100 → Cities: 2, 5, 3, 1, 4
- Individual 2: 101010001100011 → Cities: 5, 2, 1, 4, 3

A **1-point crossover (1PTX)** is applied **after the first bit**, meaning the first bit remains unchanged, and the rest of the string is swapped.



Problems After Crossover

1PTX is not a suitable crossover operator for TSP because it **breaks the permutation constraint**, leading to **duplicate and missing cities** in the offspring. Instead, order-preserving crossover methods (like PMX or OX) should be used to maintain valid city sequences.

Step 2: After Applying 1-Point Crossover

- New Individual 1: 001010001100011 → Cities: 1, 2, 2, 1, 4, 3
- New Individual 2: 110101011001100 → Cities: 6, 5, 3, 1, 2

Issues in the Result

1. Duplicate Cities Appear (Violating the Permutation Rule)

- In Individual 1, city 2 appears twice (highlighted in purple).
- In Individual 2, city 5 appears twice (highlighted in yellow).

2. Missing Cities (Permutation Violation)

- The original cities were {2, 5, 3, 1, 4} and {5, 2, 1, 4, 3}, but after crossover:
 - Individual 1 is missing city 5.
 - Individual 2 is missing city 4.

3. Invalid TSP Route

- Since the TSP requires a valid tour where each city appears exactly once, the offspring solutions are **not valid TSP routes**.



Example (The Issues)

- A valid tour for TSP must satisfy the permutation constraint, meaning cities must be represented as a *single entity* and arranged in a **specific order**.
- Binary representation cannot inherently preserve the permutation constraint, especially during crossover operations, as cities are treated as independent bits rather than whole units.
- Using *permutation-preserving operators* (e.g. **PMX** (Partially Mapped Crossover), **OX** (Order Crossover), or **CX** (Cycle Crossover) → more about them later) ensures valid solutions without requiring repair.



University of
Nottingham

UK | CHINA | MALAYSIA

Permutation- Preserving Genetic Operators



Correct Approach for TSP

- **Permutation Encoding:** Represent each individual as an ordered list of cities:
 - Example: [2,5,3,1,4].
- Use permutation-preserving crossover and mutation:
 - **Crossover:**
 - Partially Mapped Crossover (PMX)
 - Order Crossover (OX)
 - Cycle Crossover (CX).
 - **Mutation:**
 - Swap Mutation
 - Inversion Mutation
 - Scramble Mutation



Partially Mapped Crossover (PMX)

- **PMX** ensures that offspring inherit valid permutations (no duplicates or missing elements) from their parents.
- Builds offspring by
 1. choosing a subsequence of a tour from one parent.
 - a) choose two random cut points to serve as swapping boundaries.
 - b) swap segments between cut points.
 2. preserving the order and position of as many cities as possible from other parent.
- Exploits important similarities in the value and ordering simultaneously.



How PMX works

1. **Select Parents:** Choose two parent permutations.
2. **Choose Crossover Points:** Randomly select *2 crossover points* within the parent sequences.
3. **Copy Segments:** Copy the segment between the crossover points from Parent 1 to Offspring 1 and from Parent 2 to Offspring 2.
4. **Fill Remaining Positions:** Fill the remaining positions in both offspring with genes from the other parent, *avoiding duplicates*.
5. **Repair Offspring (if needed):** *Replace any duplicates* (resolve conflicts) in the offspring with corresponding values from the other parent to ensure valid permutations.



PMX Example 1

Example:

p1: (1 2 3 | 4 5 6 7 | 8 9)

p2: (4 5 2 | 1 8 7 6 | 9 3)

step 1: swap segments

o1: (x x x | 1 8 7 6 | x x)

o2: (x x x | 4 5 6 7 | x x)

this also defines mappings:

$1 \Leftrightarrow 4, 8 \Leftrightarrow 5, 7 \Leftrightarrow 6, 6 \Leftrightarrow 7$

step 2: fill in cities from other parents if no conflict

o1: (x 2 3 | 1 8 7 6 | x 9)

o2: (x x 2 | 4 5 6 7 | 9 3)

step 3: use *mappings* for conflicted positions

o1: (**4** 2 3 | 1 8 7 6 | **5** 9)

o2: (**1 8** 2 | 4 5 6 7 | 9 3)



PMX Example 1 – Step 3

Step 3 of Partially Mapped Crossover (PMX)

Step 3 of PMX ensures that all cities appear exactly **once** in each offspring by resolving **conflicts** using the **mappings** established in Step 1.

Mappings from Step 1:

The swapped segment defines conflicts where numbers are duplicated in each offspring. The mapping relationships are:

- $1 \Leftrightarrow 4$
- $8 \Leftrightarrow 5$
- $7 \Leftrightarrow 6$
- $6 \Leftrightarrow 7$

These mappings help resolve conflicts by replacing duplicate values with their mapped counterparts.



PMX Example 1 – Step 3

Applying Step 3 to Offspring 1 (o1)

State Before Step 3 (Step 2 Result)

o1: (x 2 3 | 1 8 7 6 | x 9)

- The missing values are in positions **1** and **8**.

Resolving Conflicts

Note: To resolve conflict, if direct swap works, do it first.

1. The first missing value (position 1) should be **4** (from p1). (originally should be 1 but with conflicts)
2. The last missing value (position 8) should be **5**, but 5 is already in the swapped segment.
 - Using the mapping $8 \leftrightarrow 5$, we replace **8** with **5**. (originally should be 8 but with conflicts)

Thus, o1 becomes:

✅ (4 2 3 | 1 8 7 6 | 5 9)



PMX Example 1 – Step 3

Applying Step 3 to Offspring 2 (o2)

State Before Step 3 (Step 2 Result)

o2: (x x 2 | 4 5 6 7 | 9 3)

- The missing values are in positions **1** and **2**.

Resolving Conflicts

Note: To resolve conflict, if direct swap works, do it first.

1. The first missing value (position 1) should be **1**. (originally should be 4 but with conflicts)
2. The second missing value (position 2) should be **8** (from p2). (originally should be 5 but with conflicts)

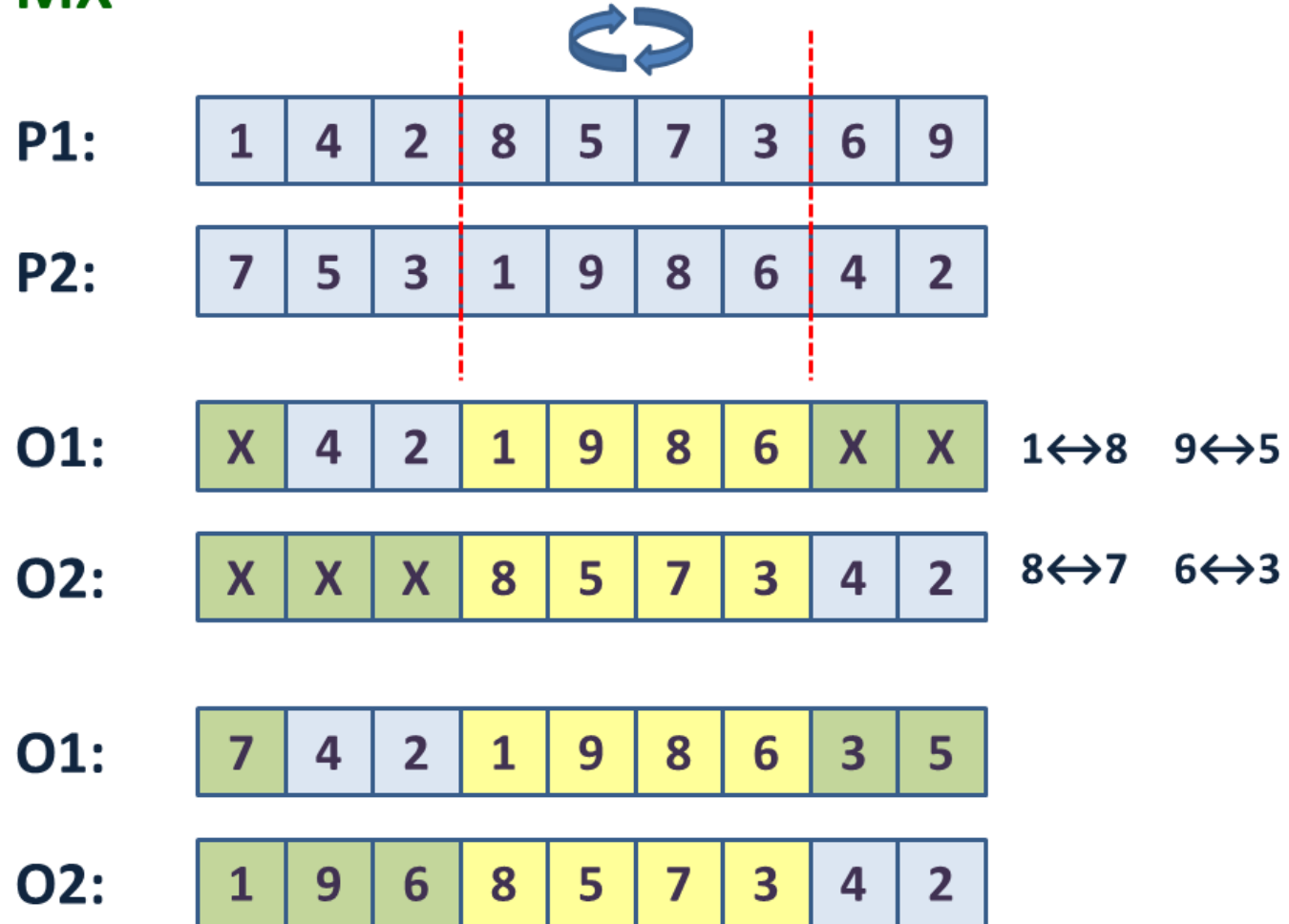
Thus, o2 becomes:

✓ (1 8 2 | 4 5 6 7 | 9 3)



PMX Example 2

PMX





PMX Example 2

Step 1: Identify Parent Sequences and Crossover Points

We are given:

- **P1:** 1 4 2 | 8 5 7 3 | 6 9
- **P2:** 7 5 3 | 1 9 8 6 | 4 2

The two crossover points are **between positions 3 and 7**, meaning the segment (8 5 7 3) from P1 swaps with (1 9 8 6) from P2.

Step 2: Swap the Segments

After swapping, we get:

- **O1 (from P1):** x x x | 1 9 8 6 | x x
- **O2 (from P2):** x x x | 8 5 7 3 | x x

At this point, some positions remain **empty (X)**.



PMX Example 2

Step 3: Identify Mappings from Swapped Segments

The swapped segments create mapping relationships:

P1 → P2	P2 → P1
8 ↔ 1	1 ↔ 8
5 ↔ 9	9 ↔ 5
7 ↔ 8	8 ↔ 7
3 ↔ 6	6 ↔ 3



PMX Example 2

Step 3: Fill in Remaining Positions

Now, we fill the missing positions while avoiding **duplicate cities**.

For O1 (was `X X X | 1 9 8 6 | X X`)

- Positions 2 and 3 remain unchanged from P1 (4 and 2), since they are **not** in the swapped segment.
- Positions 1, 8, and 9 need to be filled using P2's corresponding values first and then resolved using mappings.

Position	P1 Value	Corresponding O2 Value	Conflict?	Mapped Value (if needed)	Final O1 Value
1	1	7 (from O2)	✗ No	-	7
2	4	-	✗ No	-	4
3	2	-	✗ No	-	2
8	6	4 (from O2)	✓ Yes	6 → 3 (mapped)	3
9	9	2 (from O2)	✓ Yes	9 → 5 (mapped)	5

Note: To resolve conflict, if direct swap works, do it first.

(originally should be 6 but with conflicts. Use mapping)

(originally should be 9 but with conflicts. Use mapping)

✓ Final O1:

7 4 2 | 1 9 8 6 | 3 5



PMX Example 2

For O2 (was `X X X | 8 5 7 3 | X X`):

- The missing positions (1st, 2nd, 3rd, 8th, and 9th values) should come from P2 (`7 5 3 _ _ _ 4 2`).
- We check if they are in the swapped segment (8,5,7,3). If a conflict exists, we use the mapping.

Position	P2 Value	Already in O2?	Need Mapping?	Final O2 Value
1	7	✓ (in swapped)	Yes	1
2	5	✓ (in swapped)	Yes (5 → 9)	9
3	3	✓ (in swapped)	Yes (3 → 6)	6
8	4	✗ (not in swapped)	No	4
9	2	✗ (not in swapped)	No	2

Note: To resolve conflict, if direct swap works, do it first.

(originally should be 5 but with conflicts. Use mapping)

(originally should be 3 but with conflicts. Use mapping)

✓ Final O2:

`1 9 6 | 8 5 7 3 | 4 2`



PMX Example 3

Consider 6 cities labeled as:

$$\{A, B, C, D, E, F\}$$

Each tour (chromosome) is represented as a permutation.

Step 1: Choose Two Parent Tours

Let's take two parent permutations:

- **Parent 1:** A B C D E F
- **Parent 2:** F E D C B A



PMX Example 3

Step 2: Select Crossover Points

Assume **crossover points** are chosen at positions **2 and 4** (1-based index). That means:

- The segment **between positions 2 and 4** will be swapped.

Parent 1:

A B | C D E | F

Parent 2:

F E | D C B | A

Step 3: Swap the Segments

We swap the middle segments between parents:

- **Offspring 1 (initial swap):** A B | D C B | F
- **Offspring 2 (initial swap):** F E | C D E | A

Now, we have **invalid permutations** (duplicates exist), so we need to **fix them**.



PMX Example 3

Step 4: Resolve Duplicates using Mapping

Each swapped segment creates a **mapping** between elements. The mappings based on the swap are:

Parent 1 → Parent 2	Parent 2 → Parent 1
C → D	D → C
D → C	C → D
E → B	B → E



PMX Example 3

Now, we apply these mappings to the **remaining genes** in the offspring.

Fix Offspring 1 (A B | D C B | F) A and F has no conflict remaining in its original positions

- B is duplicated (already in the swapped part).
- Mapping shows that $B \rightarrow E$, so we replace the extra B with E.

Final Offspring 1:

✓ A E | D C B | F

Fix Offspring 2 (F E | C D E | A) F and A has no conflict remaining in its original positions

- E is duplicated.
- Mapping shows that $E \rightarrow B$, so we replace the extra E with B.

Final Offspring 2:

✓ F B | C D E | A



PMX Example 3

Final Offspring after PMX

- Offspring 1: A E D C B F
- Offspring 2: F B C D E A

These new offspring preserve **valid TSP routes** without duplicates while inheriting traits from both parents.

Summary of PMX Steps

1. Choose two parents.
2. Pick two random crossover points.
3. Swap the segments between the parents.
4. Fix duplicates using a mapping table.
5. Generate valid offspring permutations.



Order Crossover (OX)

- **Order Crossover (OX)** is a crossover operator designed to *preserve the relative order of the genes from the parent solutions* while ensuring that the offspring remain valid permutations.
- Builds offspring by
 1. choosing a subsequence of a tour from one parent.
 2. preserving *relative order of cities* from other parent.
- Exploits the property that *ordering of cities* important, not positions.
9-3-4-5-2-1-8-7-6 and
4-5-2-1-8-7-6-9-3 are identical



How OX works

1. **Select Parents:** Choose two parent permutations from the population.
2. **Choose Crossover Points:** Randomly select *two crossover points* in the parent sequences. These points define a segment that will be copied from one parent to the offspring.
3. **Copy the Segment:** Copy the segment between the two crossover points from one parent (let's call it Parent 1) to the offspring.
4. **Fill the Remaining Positions:** Fill in the remaining positions of the offspring with the genes from the other parent (Parent 2), *ensuring that the relative order of the genes in the segment is preserved*, and that no gene appears more than once.



Additional Note

- In **Order Crossover (OX1)**, **step 2 always starts from the second cutoff point**. This is because OX1 is designed to maintain relative order and position of elements from the second parent while preserving a subsequence from the first parent.
- **Can We Start from a Different Point?**
 - In **classical OX1**, **no** – The second step always begins from the second crossover point to maintain order and avoid bias.
 - In **modified versions (OX2, OX3, etc.)**, some variants may change how the remaining elements are inserted, but the general principle remains: **respect order and avoid repetition**.
- **For COMP2024/39, we shall follow OX1 – starting from second cutoff point.**



OX – Example 1

Example:

p1: (1 2 3 | 4 5 6 7 | 8 9)

p2: (4 5 2 | 1 8 7 6 | 9 3)

step 1: copy segments into offspring

o1: (x x x | 4 5 6 7 | x x)

o2: (x x x | 1 8 7 6 | x x)

step 2: **starting from 2nd cut point of one parent (based on OX1)**, cities from other parent copied in same order, omitting symbols already present; if end of string reached, continue from beginning of string sequence of cities in 2nd parent (from 2nd cut point) is

9-3-4-5-2-1-8-7-6 (remove 4,5,6,7 which are in 1st offspring)

9-3-2-1-8

place into first offspring o1: (**2 1 8** | 4 5 6 7 | **9 3**)

similarly the 2nd offspring o2: (**3 4 5** | 1 8 7 6 | **9 2**)



OX – Example 1

Steps of OX with Example

Given Parents

- P1: (1 2 3 | 4 5 6 7 | 8 9)
- P2: (4 5 2 | 1 8 7 6 | 9 3)

Step 1: Copy the Middle Segment

We choose two crossover points and copy the segment between them directly from each parent into the offspring.

- O1 (from P1 segment): (x x x | 4 5 6 7 | x x)
- O2 (from P2 segment): (x x x | 1 8 7 6 | x x)



OX – Example 1

Step 2: Fill in the Remaining Positions

The missing positions are filled using elements from the other parent **in the same order**, skipping any numbers already present in the copied segment.

1. For O1:

- Start from the **second crossover point** in P2 and proceed in order:
P2 order after the cut → (9 3 4 5 2 1 8 7 6)
- Remove numbers **already in O1** (4, 5, 6, 7)
Remaining order: 9 3 2 1 8
- Place these numbers **in the empty slots of O1** from left to right:
Final O1: (2 1 8 | 4 5 6 7 | 9 3)

2. For O2:

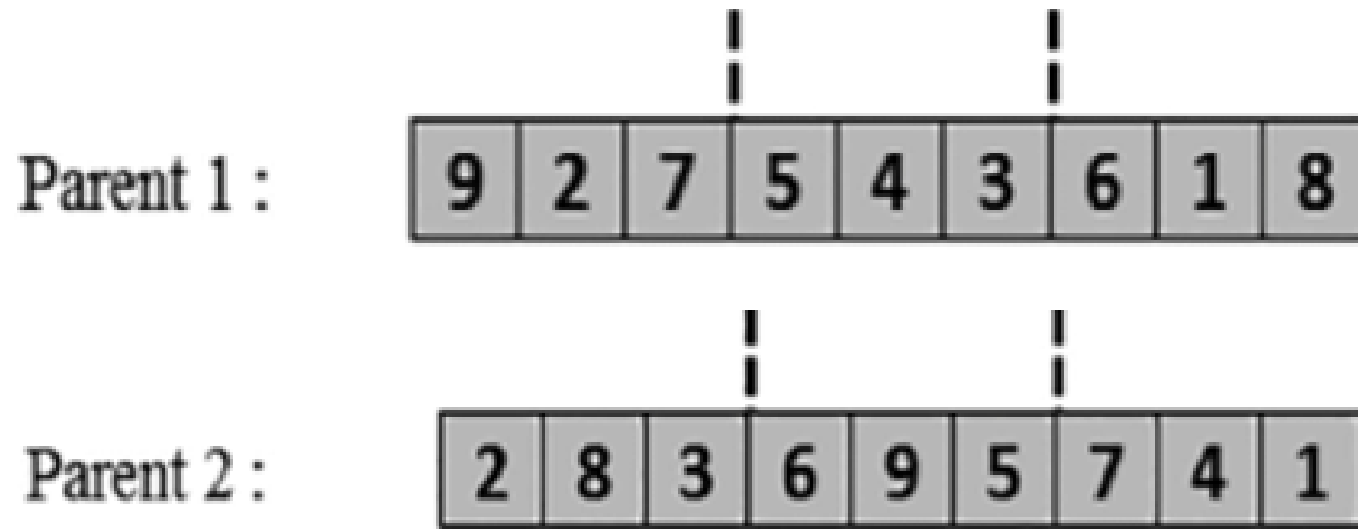
- Start from the **second crossover point** in P1 and proceed in order:
P1 order after the cut → (8 9 1 2 3 4 5 6 7)
- Remove numbers **already in O2** (1, 8, 7, 6)
Remaining order: 3 4 5 9 2
- Place these numbers **in the empty slots of O2** from left to right:
Final O2: (3 4 5 | 1 8 7 6 | 9 2)

- The middle segment is copied directly from each parent.
- The remaining numbers are filled in order from the other parent while maintaining valid permutations.



OX Example 2

Solve with OX1





OX Example 2

Step 1: Copy the middle segment into offspring

- Offspring 1 (O1) takes the middle segment from **Parent 1**:

x x x | 5 4 3 | x x x

- Offspring 2 (O2) takes the middle segment from **Parent 2**:

x x x | 6 9 5 | x x x



OX Example 2

Step 2: Fill Remaining Positions

- Take elements from **Parent 2** for **O1** and from **Parent 1** for **O2**, preserving their order **starting at the second cut point** and skipping duplicates.

Offspring 1 (O1)

1. Extract elements from Parent 2, starting from the second cutoff point

- Parent 2: 7 4 1 | 2 8 3 6 9 5
- Remove duplicates (5, 4, 3 are already in O1)
- Remaining sequence: 7 1 2 8 6 9

2. Fill remaining slots starting from the second cutoff point (position 7), then wrap around

- 8 6 9 | 5 4 3 | 7 1 2

✓ Final O1: (8 6 9 | 5 4 3 | 7 1 2)



OX Example 2

Offspring 2 (O2)

1. Extract elements from Parent 1, starting from the second cutoff point

- Parent 1: 6 1 8 | 9 2 7 5 4 3
- Remove duplicates (6, 9, 5 are already in O2)
- Remaining sequence: 1 8 2 7 4 3

2. Fill remaining slots starting from the second cutoff point (position 7), then wrap around

- 7 4 3 | 6 9 5 | 1 8 2

✓ Final O2: (7 4 3 | 6 9 5 | 1 8 2)



OX Example 3

Let's consider 8 cities labeled:

$$\{A, B, C, D, E, F, G, H\}$$

Each tour (chromosome) is represented as a permutation.

Step 1: Choose Two Parent Tours

- Parent 1: A B C D E F G H
- Parent 2: H G F E D C B A



OX Example 3

Step 2: Select Crossover Points

Assume **crossover points** are chosen at positions **3 and 6** (1-based index). That means:

- The segment **between positions 3 and 6** will be copied to the offspring.

Parent 1:

A B | C D E F | G H

Parent 2:

H G | F E D C | B A

Step 3: Copy the Segment from Parent 1 to Offspring

- Copy **C D E F** from **Parent 1** into the corresponding position in **Offspring 1**.
- Copy **F E D C** from **Parent 2** into the corresponding position in **Offspring 2**.

Offspring 1 (initial): _ _ C D E F _ _

Offspring 2 (initial): _ _ F E D C _ _



OX Example 3

Step 4: Fill in Remaining Cities in the Order of Parent 2

We now take the **remaining cities** from Parent 2 (excluding the copied ones) in the order they appear and fill the gaps.

For Offspring 1

- Parent 2: H G F E D C B A
- Remove already copied C, D, E, F → H G B A
- Fill in the blanks left to right:

Final Offspring 1: H G C D E F B A

For Offspring 2

- Parent 1: A B C D E F G H
- Remove already copied F, E, D, C → A B G H
- Fill in the blanks left to right:

Final Offspring 2: A B F E D C G H



OX Example 3

Final Offspring after OX

- Offspring 1: H G C D E F B A
- Offspring 2: A B F E D C G H

These new offspring **preserve the order** of elements while inheriting traits from both parents.

Summary of OX Steps

1. Choose two parents.
2. Pick two random crossover points.
3. Copy the segment from Parent 1 to Offspring 1, and from Parent 2 to Offspring 2.
4. Fill remaining positions using the order of the other parent (excluding copied elements).
5. Generate valid offspring permutations.



Cycle Crossover (CX)

- **Cycle Crossover (CX)** works by identifying cycles of genes in the two parent chromosomes and swapping them between the parents. It ensures that the offspring maintain the *consistency of the permutation and avoids gene duplication*.
- The offspring inherit genes from each parent based on their *cycles*, preserving the relationship between positions in the parents.
- Builds offspring by
 1. choosing each city and its position from one of the parents.
 2. and when a cycle is completed, the remaining cities filled in from the other parent.
- Preserves *absolute positions* of the elements in the parent sequence.



How CX Works

1. **Parent Chromosomes:** Two parent chromosomes are selected for crossover.
2. **Identify Cycles:** Starting from the first gene in the first parent, identify a cycle of corresponding genes between the two parents. A cycle is formed by selecting a gene from one parent, finding its corresponding gene in the other parent, and repeating this process until the starting gene is reached again.
3. **Create Offspring:** Genes from the first parent are placed in the corresponding positions in the offspring based on the cycles identified. Genes from the second parent fill the remaining positions in the offspring.
4. **Repeat for All Cycles:** Repeat the cycle identification and offspring generation until all positions in the offspring are filled.



Cycle Crossover (CX) Example 1

p1: (1 2 3 4 5 6 7 8 9)

p2: (4 1 2 8 7 6 9 3 5)

o1: (1 x x x x x x x x)

o1: (1 x x 4 x x x x x)

o1: (1 x x 4 x x x 8 x)

o1: (1 x 3 4 x x x 8 x)

o1: (1 2 3 4 x x x 8 x)

2 requires selection of 1 causing a cycle, so rest filled from other parent:

o1: (1 2 3 4 7 6 9 8 5)

similarly

o2: (4 1 2 8 5 6 7 3 9)

Randomly select a starting point(city) in p1: 1

Copy cities from p1 until a cycle is obtained while mapping from p1 to each corresponding city in p2



CX Example 1

Step 1: Identify the First Cycle

We start with $P1[1] = 1$ and trace the cycle using $P2$.

1. $P1[1] = 1 \rightarrow P2[1] = 4 \rightarrow$ Go to index **4** in $P1$.
2. $P1[4] = 4 \rightarrow P2[4] = 8 \rightarrow$ Go to index **8** in $P1$.
3. $P1[8] = 8 \rightarrow P2[8] = 3 \rightarrow$ Go to index **3** in $P1$.
4. $P1[3] = 3 \rightarrow P2[3] = 2 \rightarrow$ Go to index **2** in $P1$.
5. $P1[2] = 2 \rightarrow P2[2] = 1$ (Back to the start, cycle complete).

✓ Cycle 1 indices: [1, 2, 3, 4, 8]

✓ Copy these elements from $P1$ to $O1$

✓ Copy the remaining elements from $P2$ to $O1$

$O1$ after Cycle 1: ✓ (1 2 3 4 x x x 8 x)

Step 2: Fill Remaining Positions from $P2$

Now, fill the missing positions using $P2$.

✓ $O1$: (1 2 3 4 7 6 9 8 5)



CX Example 1

Step 1: Identify the First Cycle

We start with $P1[1] = 1$ and trace the cycle using $P2$.

1. $P1[1] = 1 \rightarrow P2[1] = 4 \rightarrow$ Go to index 4 in P1.
2. $P1[4] = 4 \rightarrow P2[4] = 8 \rightarrow$ Go to index 8 in P1.
3. $P1[8] = 8 \rightarrow P2[8] = 3 \rightarrow$ Go to index 3 in P1.
4. $P1[3] = 3 \rightarrow P2[3] = 2 \rightarrow$ Go to index 2 in P1.
5. $P1[2] = 2 \rightarrow P2[2] = 1$ (Back to the start, cycle complete).

✓ Cycle 1 indices: [1, 2, 3, 4, 8]

✓ Copy these elements from $P2$ to O_2

✓ O_2 after Cycle 1: (4 1 2 8 x x x 3 x)

Step 2: Fill Remaining Positions from P1

Now, fill the missing positions using $P1$.

1. Position 5 $\rightarrow$ From $P1$: 5
2. Position 6 $\rightarrow$ From $P1$: 6
3. Position 7 $\rightarrow$ From $P1$: 7
4. Position 9 $\rightarrow$ From $P1$: 9

✓ Final O_2 : (4 1 2 8 5 6 7 3 9)



CX Example 2

Step 1: Given Parents

We have the following parent tours (permutations):

- Parent 1: A B C D E F
- Parent 2: B C D A F E

Our goal is to apply Cycle Crossover (CX) to produce two valid offspring.



CX Example

Step 2: Identify Cycles

CX operates by identifying **cycles** where elements maintain their **relative positions**.

Find the First Cycle

1. Start at **position 1 (index 0)** in **Parent 1** → **A**
2. Look at **Parent 2's position 0** → **B**
3. Find **B** in **Parent 1** → It appears at **position 1**
4. Look at **Parent 2's position 1** → **C**
5. Find **C** in **Parent 1** → It appears at **position 2**
6. Look at **Parent 2's position 2** → **D**
7. Find **D** in **Parent 1** → It appears at **position 3**
8. Look at **Parent 2's position 3** → **A**
9. We are back at the starting element **A**, so the **cycle ends**.

Cycle Elements

- **Cycle 1 Positions:** [0, 1, 2, 3] → **A, B, C, D**

Evolutionary Algorithm II



CX Example

Step 3: Generate Offspring

- Offspring 1: Keep cycle elements from Parent 1, fill remaining from Parent 2.
- Offspring 2: Keep cycle elements from Parent 2, fill remaining from Parent 1.

Position	Parent 1	Parent 2	Cycle?	Offspring 1 (from P1)	Offspring 2 (from P2)
0	A	B	✓ Cycle 1	A	B
1	B	C	✓ Cycle 1	B	C
2	C	D	✓ Cycle 1	C	D
3	D	A	✓ Cycle 1	D	A
4	E	F	✗ Not in cycle	F (from P2)	E (from P1)
5	F	E	✗ Not in cycle	E (from P2)	F (from P1)



CX Example

Step 4: Final Offspring After CX

- Offspring 1: A B C D F E ✓
- Offspring 2: B C D A E F ✓

These offspring maintain valid permutations while preserving relative order through cycles.

Summary of Cycle Crossover (CX)

1. Find cycles by tracing elements' positions from **Parent 1** → **Parent 2**.
2. Copy cycle elements directly into the offspring.
3. Fill remaining elements from the opposite parent.
4. Ensure valid permutations (no duplicates, all elements included).



PMX, OX and CX

Crossover Type	Structure Preserved	Order Preserved
PMX	Position-based	Partially
OX	Order-based	Fully
CX	Cycle-based	Fully



More Genetic Operators – Crossover for TSP

Operator Name	Date	Authors
Alternating Position Crossover (AP)	(1999)	Larranaga, Kuijpers, Poza, Murga
Cycle Crossover (CX)	(1987)	Oliver, Smith and Holland
Distance Preserving Crossover (DPX)	(1996)	Freisbein and Merz
Edge Assembly Crossover (EAX)	(1997)	Nagata and Kobayashi
Edge Recombination Crossover (ER)	(1989)	Whitley, Timothy, Fuquay
Heuristic Crossover (HEU)	(1987)	Grefenstette
Inver-over Operator (IOO)	(1998)	Tao and Michalewicz
Maximal Preservative Crossover (MPX)	(1988)	Mühlenbein, Schleuter and Krämer
Position Based Crossover (POS)	(1991)	Syswerda
Order Crossover (OX1)	(1985)	Davis
Order Based Crossover (OX2)	(1991)	Syswerda
Partially mapped Crossover (PMX)	(1985)	Goldberg and Lingle
Voting Recombination Crossover (VR)	(1989)	Mühlenbein
Natural Crossover (NX)	(2000)	Jung and Moon
Voronoi Quantized Crossover (VQX)	(2002)	Seo and Moon



More Genetic Operators – Crossover for TSP

- Alternating Position Crossover (AP): Larrañaga, P., Kuijpers, C. M. H., Poza, M., & Murga, R. H. (1999). Genetic algorithms for the travelling salesman problem. A review of representations and operators. *Artificial Intelligence Review*, 13(2), 129-170.
- **Cycle Crossover (CX):** Oliver, I. M., Smith, D. J., & Holland, J. R. C. (1987). **A study of permutation crossover operators on the traveling salesman problem. In Proceedings of the second international conference on genetic algorithms (pp. 224-230).**
- Distance Preserving Crossover (DPX): Freisleben, B., & Merz, P. (1996). A new crossover operator for the traveling salesman problem. In *International Conference on Parallel Problem Solving from Nature* (pp. 523-532). Springer, Berlin, Heidelberg.
- Edge Assembly Crossover (EAX): Nagata, Y., & Kobayashi, S. (1997). A fast and efficient edge assembly crossover for the traveling salesman problem. In *Proceedings of the seventh international conference on genetic algorithms* (pp. 450-457). Morgan Kaufmann Publishers Inc.
- Edge Recombination Crossover (ER): Whitley, D., Starkweather, T., & Fuquay, D. (1989). Scheduling problems and traveling salesmen: The genetic edge recombination operator. In *Proceedings of the third international conference on genetic algorithms* (pp. 133-140). Morgan Kaufmann Publishers Inc.
- Heuristic Crossover (HEU): Grefenstette, J. J. (1987). Incorporating problem specific knowledge into genetic algorithms. In *Genetic algorithms and simulated annealing* (pp. 42-60). Morgan Kaufmann.
- Inver-over Operator (IOO): Tao, G., & Michalewicz, Z. (1998). Inver-over operator for the travelling salesman problem. In *Parallel Problem Solving from Nature—PPSN V* (pp. 803-812). Springer, Berlin, Heidelberg.
- Maximal Preservative Crossover (MPX): Mühlenbein, H., Gorges-Schleuter, M., & Krämer, O. (1988). Evolution algorithms in combinatorial optimization. *Parallel Computing*, 7(1-3), 65-85.



More Genetic Operators – Crossover for TSP

- **Position Based Crossover (POS):** Syswerda, G. (1991). Schedule optimization using genetic algorithms. In Genetic algorithms: Proceedings of the fourth international conference on genetic algorithms and their applications (pp. 332-339). Morgan Kaufmann Publishers Inc.
- **Order Crossover (OX1):** Davis, L. (1985). Applying adaptive algorithms to epistatic domains. In Proceedings of the ninth international joint conference on artificial intelligence (Vol. 2, pp. 62-69). Morgan Kaufmann Publishers Inc.
- **Order Based Crossover (OX2):** Syswerda, G. (1991). Schedule optimization using genetic algorithms. In Genetic algorithms: Proceedings of the fourth international conference on genetic algorithms and their applications (pp. 332-339). Morgan Kaufmann Publishers Inc. (Same as POS)
- **Partially Mapped Crossover (PMX):** Goldberg, D. E., & Lingle, R. (1985). Alleles, loci, and the traveling salesman problem. In Proceedings of an international conference on genetic algorithms and their applications (pp. 154-159). Carnegie-Mellon University.
- **Voting Recombination Crossover (VR):** Mühlenbein, H. (1989). Parallel genetic algorithms, population genetics and combinatorial optimization. In Proceedings of the third international conference on genetic algorithms (pp. 416-421). Morgan Kaufmann Publishers Inc.
- **Natural Crossover (NX):** Jung, S., & Moon, B. R. (2000). A hybrid genetic algorithm for the traveling salesman problem. In Proceedings of the Genetic and Evolutionary Computation Conference (pp. 1425-1432). Morgan Kaufmann Publishers Inc.
- **Voronoi Quantized Crossover (VQX):** Seo, J., & Moon, B. R. (2002). A Voronoi quantized crossover for the traveling salesman problem. In Proceedings of the Genetic and Evolutionary Computation Conference (pp. 1041-1048). Morgan Kaufmann Publishers Inc.



University of
Nottingham

UK | CHINA | MALAYSIA

Mutation Operators for TSP



Mutation Operators for TSP

<u>Operator Name</u>	<u>Date</u>	<u>Authors</u>
Displacement Mutation (DM)	(1992)	Michalewicz
Exchange Mutation (EM)	(1990)	Banzhaf
Insertion Mutation (ISM)	(1988)	Fogel
Inversion Mutation (IVM)	(1990)	Fogel
Scramble Mutation (SM)	(1991)	Syswerda
Simple Inversion Mutation (SIM)	(1975)	Holland



Mutation Operators for TSP

- Displacement Mutation (DM): Michalewicz, Z. (1992). Genetic algorithms + data structures = evolution programs. Springer Science & Business Media.
- **Exchange Mutation (EM): Banzhaf, W. (1990). The "molecular" traveling salesman. Biological Cybernetics, 64(1), 7-14.**
- **Insertion Mutation (ISM): Fogel, D. B. (1988). An evolutionary approach to the traveling salesman problem. Biological Cybernetics, 60(2), 139-144.**
- Inversion Mutation (IVM): Fogel, D. B. (1990). A parallel processing approach to a multiple traveling salesman problem using evolutionary programming. In Proceedings of the Fourth Annual Symposium on Parallel Processing (Vol. 1, pp. 318-326). IEEE Computer Society.
- Scramble Mutation (SM): Syswerda, G. (1991). Schedule optimization using genetic algorithms. In Genetic algorithms: Proceedings of the fourth international conference on genetic algorithms and their applications (pp. 332-339). Morgan Kaufmann Publishers Inc.
- Simple Inversion Mutation (SIM): Holland, J. H. (1975). Adaptation in natural and artificial systems: An introductory analysis with applications to biology, control, and artificial intelligence. University of Michigan Press.



Mutation Operator for TSP

- Both **Exchange Mutation (EM)** and **Insertion Mutation (IM)** are mutation operators used to introduce *diversity* in solutions by modifying the order of cities in the TSP path. They work differently in how they rearrange the sequence.

Feature	Exchange Mutation (EM)	Insertion Mutation
Operation	Swaps two cities in the path.	Moves one city to a different position.
Degree of Change	Small, localized.	Medium, with more significant changes.
Exploration	Limited exploration of solution space.	Broader exploration of solution space.
Efficiency	Better for fine-tuning solutions.	Better for introducing diversity.
Complexity	Simple to implement.	Slightly more complex.



Exchange Mutation (EM) for TSP

- **Exchange Mutation** introduces diversity into the population by *swapping two cities* (genes) in the path.
- The overall order remains the same except for the exchanged positions.
 - 1. Select Two Cities:** Randomly choose two cities (positions) in the path.
 - 2. Swap the Cities:** Exchange the positions of these two cities in the sequence.
 - 3. Form New Path:** The rest of the path remains unchanged.



EM Example

- Suppose the initial path is:
 - **Original Path:** [1, 2, 3, 4, 5, 6, 7]

- **Steps:**
 1. Randomly select two positions, say position 2 (city 2) and position 5 (city 5).
 2. Swap the cities at these positions.
- **Mutated Path:** [1, 5, 3, 4, 2, 6, 7]



Insertion Mutation (ISM) for TSP

- **Insertion Mutation** modifies a path by *moving one city to a different position* in the sequence.
- This allows for more structural changes compared to Exchange Mutation.
 - 1. Select a City:** Randomly choose one city (position) in the path.
 - 2. Remove the City:** Temporarily remove the city from the sequence.
 - 3. Insert the City:** Insert the city into another randomly selected position.
 - 4. Form New Path:** Adjust the sequence to maintain validity.



ISM Example

- Suppose the initial path is:
 - **Original Path:** [1, 2, 3, 4, 5, 6, 7]
- **Steps:**
 1. Randomly select a city, say city 5 at position 5.
 2. Remove city 5 from the sequence.
 - New sequence: [1, 2, 3, 4, __, 6, 7]
 3. Insert city 5 into another position, say position 2.
 - **Mutated Path:** [1, 5, 2, 3, 4, 6, 7]



Replacement techniques used in GAs for the TSP

- Similar to what we discussed in Lecture 5 for MAX-SAT problem

Technique	Exploration	Exploitation	Risk of Premature Convergence	Efficiency
Generational Replacement	High	Low	High	High
Steady-State Replacement	Medium	Medium	Low	Medium
Elitism Replacement	Low	High	High (if elites dominate)	High
Fitness-Based Replacement	Medium	High	Low	Medium
Age-Based Replacement	Medium	Low	Low	Medium
Random Replacement	High	Low	Low	High
Hybrid Replacement	Balanced	Balanced	Low	Medium



Sample Experiment Set up: GA vs MA for Solving TSP – Components

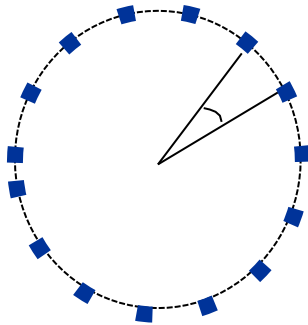
- **Representation:** *path representation* (permutation)
 - For example, (1 3 5 2 6 4) represents a tour starting from city 1, visiting 3, 5, 2, 6, 4 in that order and returning back to 1.
- **Fitness function:** path length (distance)
- **Crossover:** PMX, 2PTX (with repair/patch-up), OX1
- **Mutation Operators:** ISM, EM
- **Mate Selection:** RANK, TOUR
- **Replacement:** Truncation Selection (TG), Steady-State Replacement (SS)
- **Hill Climbing:** RMHC using ISM, RMHC using EM → **Memetic Algorithm (MA)**

A hill climbing step is applied, as long as, max. number of iterations is not exceeded and the individual is improved.

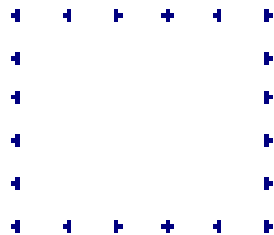
Note: Sample experiment set up reported in a paper



Experimental Data for TSP

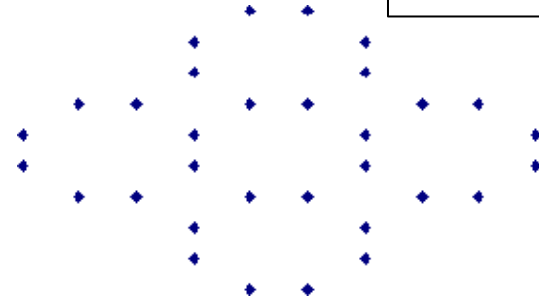


C20/30/40

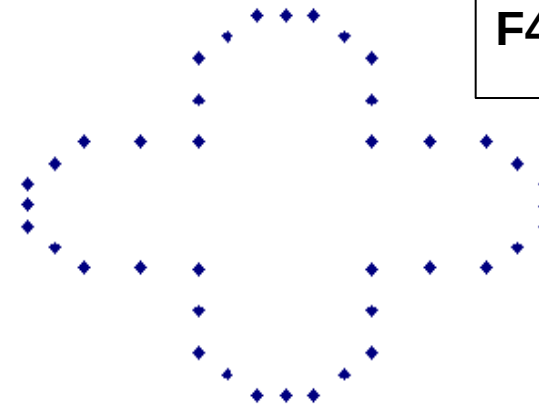


S21

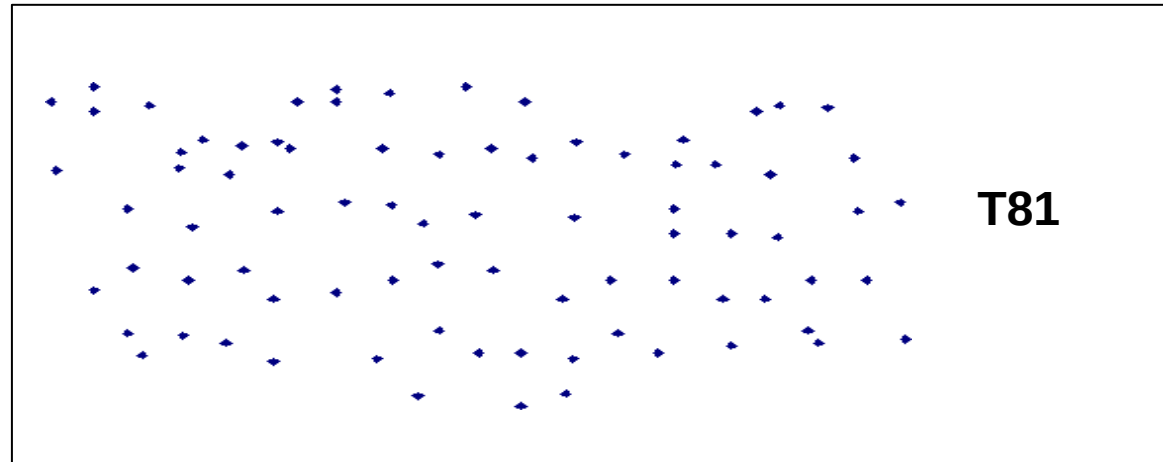
F32



F41



T81





Experimental Data

- Each dataset is labeled with a specific identifier (e.g., C20/30/40, F32, F41, S21, T81), which refers to specific TSP instances or benchmarks.
- The different shapes or arrangements of the blue dots suggest varying complexity and distance metrics between cities. Some may form circular patterns, while others appear more random or clustered.
- This variability is crucial for testing the performance and effectiveness of the algorithms, as different city arrangements can lead to different challenges in finding the optimal route.



Experimental Data for TSP

- **C20/30/40: Croes Problems** (also sometimes referred to as "Croes Data").
 - **Reasoning:** The "C" prefix suggests Croes. These are very common, simple, and often used as introductory or illustrative datasets. The numbers (20, 30, 40) indicate the number of cities in each instance. The circular arrangement in the image fits the typical Croes problem layout.
 - **Characteristics:** Cities are usually arranged in a regular pattern, often a circle or a grid.
- **F32 & F41: Falkenauer's Random Problems**
 - **Reasoning:** The "F" prefix is an indicator of Falkenauer. These are known for their more complex, clustered structures.
 - **Characteristics:** Visually, the images show a clustered pattern, which is typical of Falkenauer's instances. The numbers (32, 41) represent the number of cities.



Experimental Data for TSP

■ T81: Taillard's Random Problems

- **Reasoning:** The "T" prefix suggests Taillard's datasets. These are known for their larger sizes and more challenging, randomly generated instances.
- **Characteristics:** The image shows a large, scattered distribution of points, which aligns with the characteristics of Taillard's random problems. The number (81) represents the number of cities.

■ S21: Could be a custom-generated or simplified instance.

- **Reasoning:** The "S" prefix is not a standard prefix for well-known TSP dataset collections. The square arrangement suggests it might be a simplified or custom-generated instance, possibly for educational or demonstration purposes.
- **Characteristics:** The image shows a perfect square arrangement of points.



Results

From the best to worst, considering the average fitness per generation at each run (100 runs).

- Crossover: **OX1**, 2PTX, PMX
- Mutation: **EM**, ISM (ISM is better in finding the best), 1/chromosome - length
- Mate Selection: **TOUR** (tour-size:2), RANK
- Replacement: **TG** (keep top 2 and replace the rest with offspring, $g = \text{popSize} - 2$), SS (replace the worst pair of individuals with the best pair among the offspring and parents)
- Approach: **MA** (Hill Climbing: **EM**, ISM), GA

Note: Sample experiment result with the bolded as preferred implementation option that yield better results as reported in a research paper. For your reference.



Break





University of
Nottingham

UK | CHINA | MALAYSIA

Multi-meme Memetic Algorithms



A Classification of Memetic Algorithms

Adaptive Type		Adaptive Level		
		External	Local	Global
Static		Basic meta-Lamarckian learning / Simplerandom		
	Qualitative Adaptation		Randomdescent / Randompermddescent	Tabu-search
	Quantitative Adaptation		Sub-Problem Decomposition/ Greedy	Straightchoice/ Rankedchoice/ Roulettechoice/ Decompchoice/ Biased Roulette Wheel
Self-Adaptive			Multi-memes/ Co-evolution MA	

Classification of Memetic Algorithms (MAs) based on their adaptive type and adaptive level.

Yew-Soon Ong; Meng-Hiot Lim; Ning Zhu; Kok-Wai Wong, "Classification of adaptive memetic algorithms: a comparative study," *Systems, Man, and Cybernetics, Part B: Cybernetics, IEEE Transactions on*, vol.36, no.1, pp.141,152, Feb. 2006 [\[PDF\]](#)



A Classification of Memetic Algorithms

- **Static MAs** are simple and easy to implement but lack flexibility.
- **Adaptive MAs** improve efficiency by adjusting search strategies dynamically.
- **Self-Adaptive MAs** offer the most advanced approach by evolving strategies without external control, making them highly efficient and effective for complex optimization problems.



Adaptive Level

- **External:** Adaptation happens outside the core search process
- **Local:** Adaptation occurs within the local search phase
- **Global:** Adaptation is integrated into a higher-level global optimization strategy



Adaptive Type

- **Static:** No adaptation of learning strategies during the algorithm run.
- **Adaptive:** Adjusts learning strategies based on observed performance. It is further divided into:
 - **Qualitative Adaptation:** Uses simple rules or heuristics to modify behavior.
 - **Quantitative Adaptation:** Uses numerical adjustments to fine-tune the learning process.
- **Self-Adaptive:** The algorithm autonomously modifies its parameters or strategies without explicit control.



Self Adaptation for Genetic Operators

- **Self Adaptation:** Deciding which operators (like *crossover* and *mutation*) (during runtime) and settings to use on the fly whenever needed receiving feedback during the evolutionary search process.
- *Davis* used varying probabilities of applying different operators.
 - Uses the performance of the operator within the last few generations to update probabilities.
- *Grefenstette, Kakuza, Friesleben, Hartfelder* used subpopulations, each having different set of parameters and operators.
- *Spears* used an additional bit to decide whether apply 2- PTX or UX to an individual (co-evolution).



Self Adaptation for Genetic Operators

- Davis, L. (1989). Adapting operator probabilities in genetic algorithms. *Proceedings of the Third International Conference on Genetic¹ Algorithms (ICGA)*, 61-69
- Grefenstette, J. J. (1986). Genetic algorithms in noisy environments.
- Spears, W. M. (1995). Adapting crossover in evolutionary algorithms.



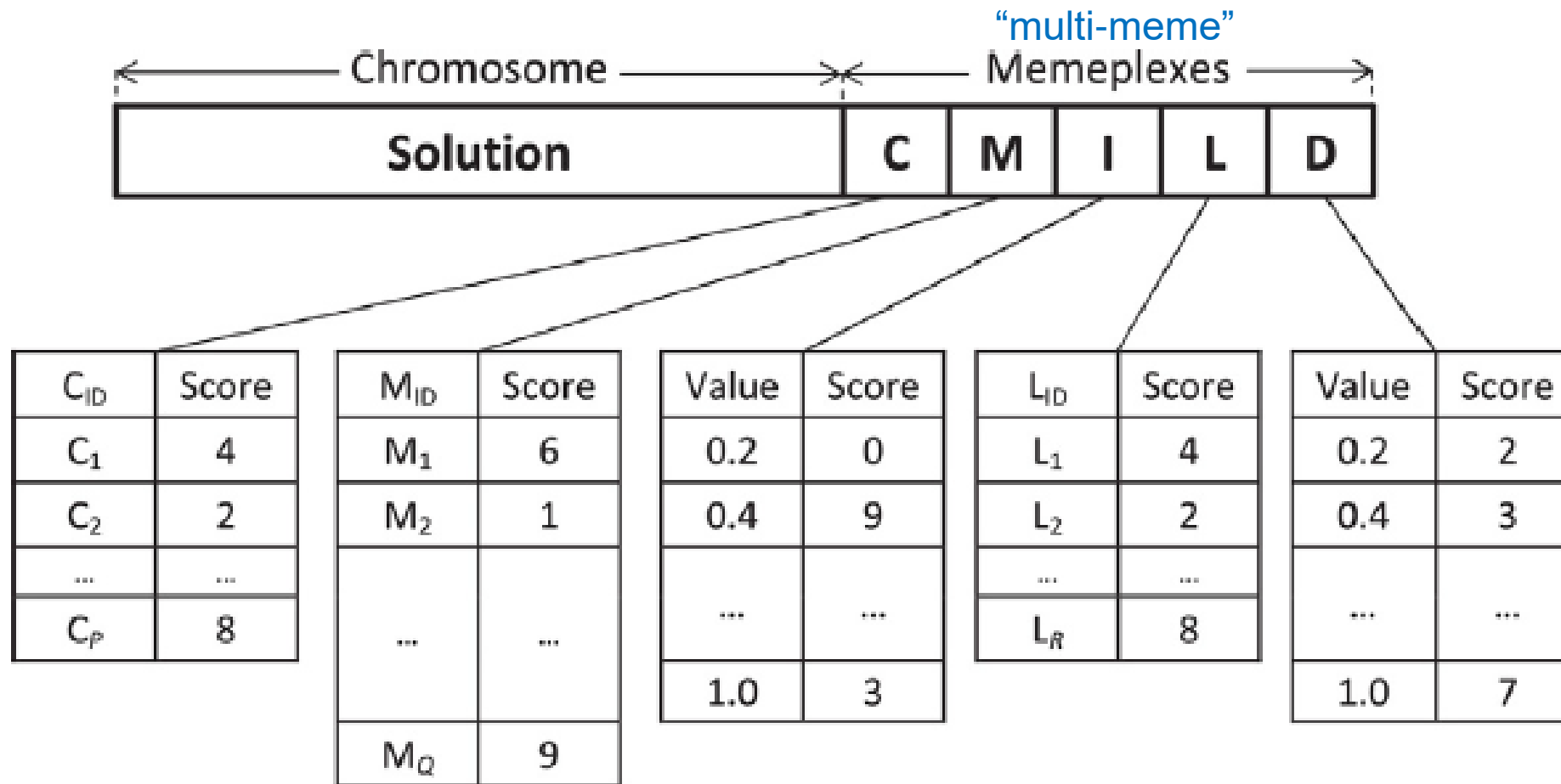
Multimeme Memetic Algorithms

- Introduce **memetic material** for each individual.
 - **Co-evolve** genetic (solution) and *memetic material*
- Krasnogor formalised in his PhD thesis*.
- A **meme** encodes how to apply an operator (which one to apply, max. no. of iterations, acceptance criteria), when to apply, where to apply, and how frequent to apply.
- Each meme directly *encodes the choices of algorithmic components to use*.
- Meme of each operator can be combined under a **memplex**.

*Krasnogor N (2002) Studies on the theory and design space of memetic algorithms, PhD Thesis, University of the West of England, Bristol, UK



Encoding of an individual Example: Genetic and Memetic material



Note: Each entry holds the utility score(or option value) for each *algorithmic component choice (parameter)*



Multimeme Memetic Algorithms

- The **Multimeme Memetic Algorithm (MMA)** is a variation of the standard memetic algorithm (MA), where individual solutions in the population are not only associated with their genes (the solution representation) but also with an *additional "meme" that represents a specific local search strategy or heuristic*.
- These **memes** evolve alongside the genetic material, and they can adapt over time to improve the algorithm's efficiency.
- This enables the algorithm to *dynamically explore and exploit* the search space by adjusting its heuristics.



Key Components of a Multimeme Memetic Algorithm

- **Genes (Solution Representation):**
 - The main representation of a solution (e.g., a permutation of cities in the TSP).
- **Mememes (Local Search Heuristics) (“memetic”):**
 - Each individual in the population is associated with a *meme*, which specifies a local search method or tuning parameter.
 - Mememes guide how *local improvements* are made for the solution.



Key Components of a Multimeme Memetic Algorithm

- **Evolution of Memes (algorithmic parameters):**
 - Mememes co-evolve with the solution population. When individuals undergo crossover or mutation, their memes may also change.
 - Memes can be inherited, mutated, or adapted over generations.
- **Local Search Integration:**
 - After crossover and mutation, a *local search* is applied to the offspring, guided by the individual's associated meme.



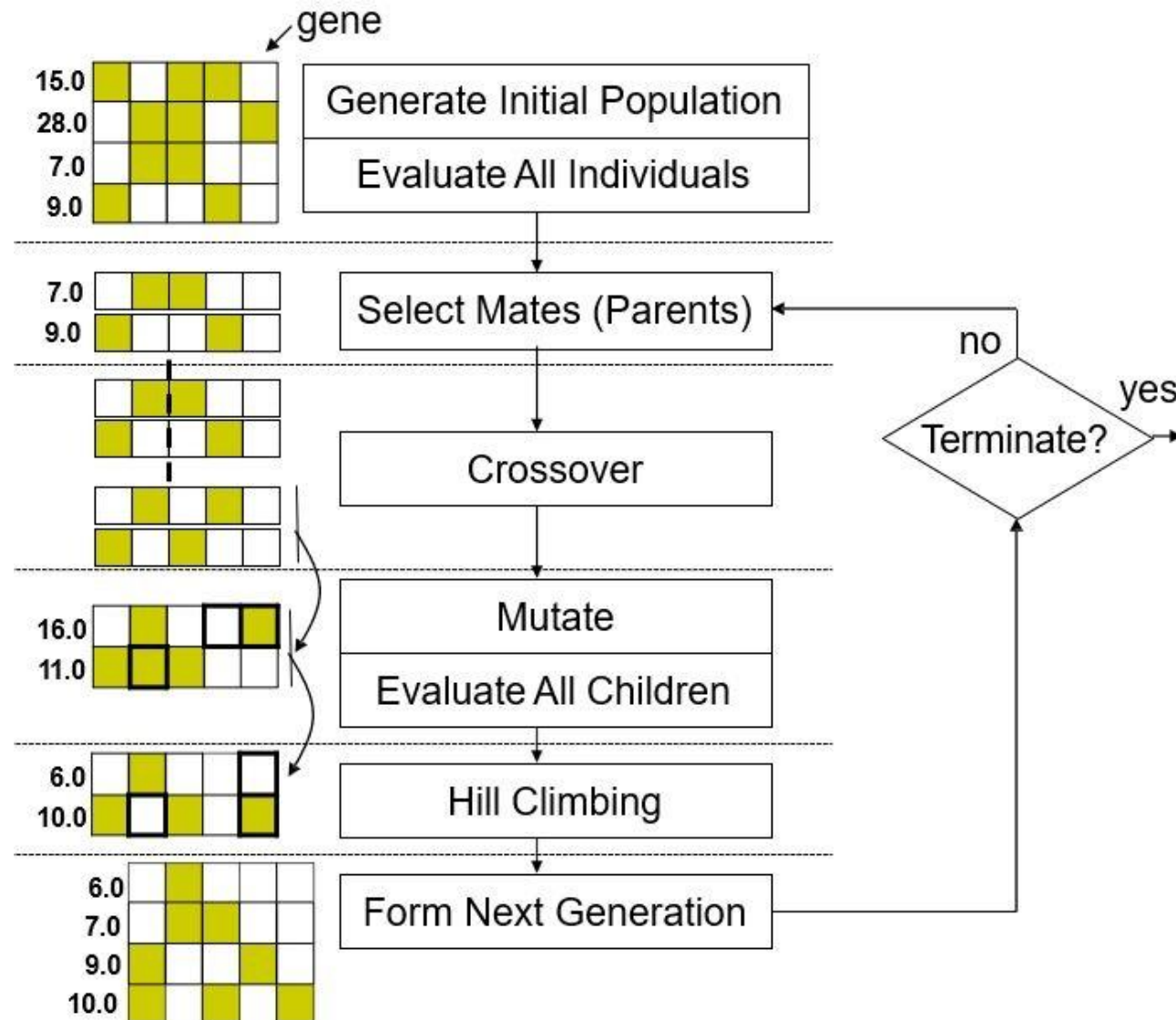
Grammar for a Memeplex– Compound of Memes

- Memeplex = <Meme>;<Meme>:<Memeplex>
- Meme = (< Where >, < When >, < How >, < Frequency >)
- Where = < Location > Crossover ; < Location > Mutation;
- Location = After ; Before
- When = < BooleanCondition >
- How = GeneralHillClimber < Move >< Strategy >< Iterations >;
- BoltzmanAdaptiveHillClimber < Move >< Strategy >< Iterations > ...
- Frequency = Always; For < Iterations >; Never ...
- Move = 2-exchange, 3-exchange
- Strategy = NextAscent; SteepestAscent; MiniMax < Size >; MaxiMin < Size >
- Size = < Num >
- Iterations = < Num >

Example of atomic setting parameters and operators as memes (from a research paper)

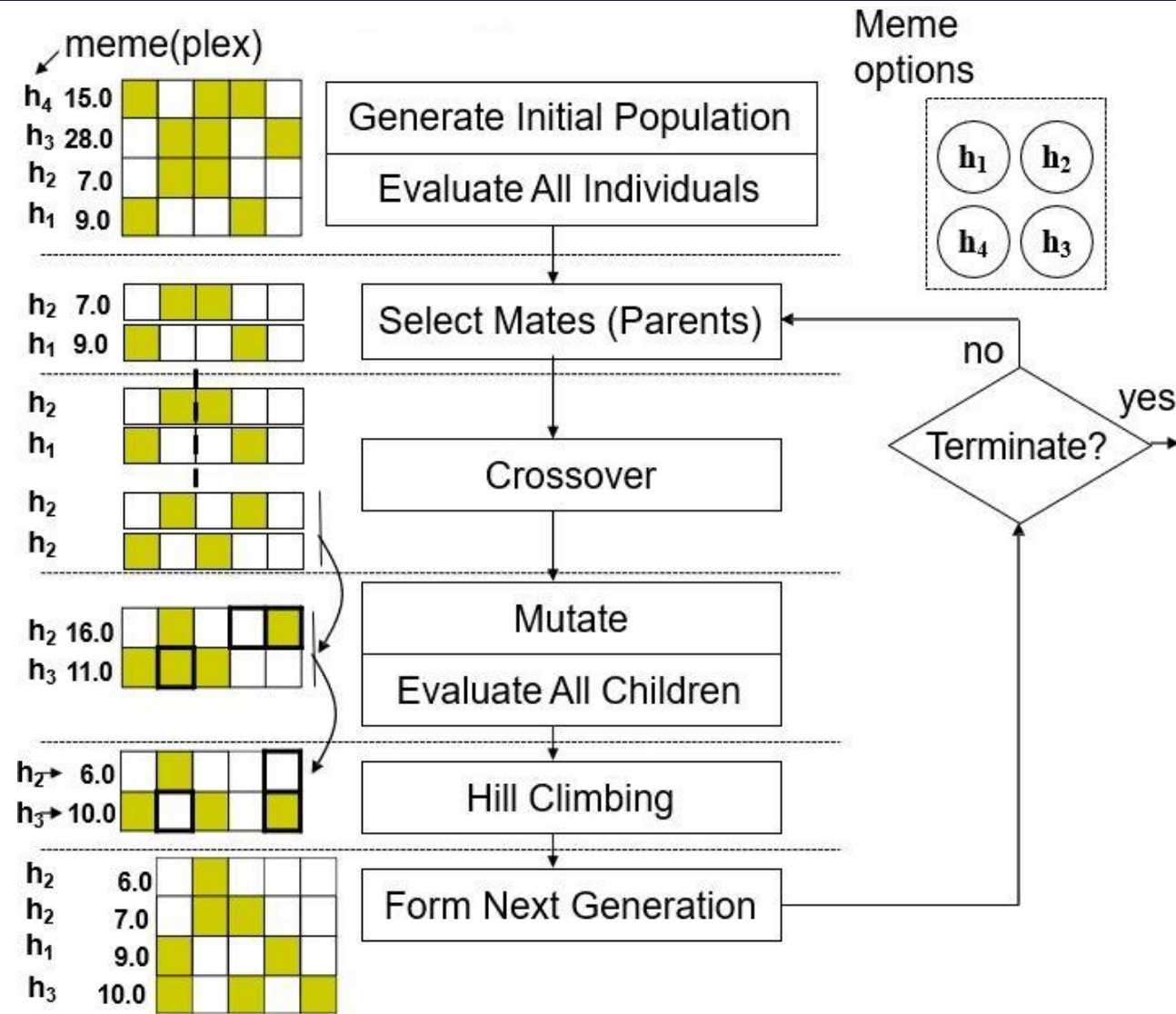


Memetic Algorithms (MAs)





Multimeme Memetic Algorithms (MMAs)





Multimeme Memetic Algorithms – Features

- Memes represent *instructions for self-improvement*.
 - Specify set of rules, programs, heuristics, strategies, behaviors, etc.
- Interaction between memes (atomic discrete instructions) and genes (solutions) are not direct.
 - Mediated by individual (genetic and memetic material)
- Memes can evolve, change by means of nontraditional genetic transformations and metrics.



Pseudocode of Multimeme Memetic Algorithms(MMA)

Algorithm 1. Pseudocode of Multimeme Memetic Algorithms (MMA) [35]

```
1:      Create a population of popSize random individuals.
2:      Apply a random hill climber to each individual
3:      while termination criteria is not satisfied do
4:          Parent1 ← Select-Parent(population, tour-size)
5:          Parent2 ← Select-Parent(population, tour-size)
6:          if Parent2 has a better fitness than Parent1 then
7:              Swap(Parent1, Parent2)
8:          end if
9:          Child ← ApplyC(Parent1crossover, Parent1, Parent2)
10:         Copy the meme of Parent1 to Child
11:         Child ← ApplyM(Childmutation, Childintensity, Child)
12:         Mutate meme of Child with a probability of IR
13:         Child ← ApplyL(Childhillclimber, Childdepth, Child)
14:         Child replaces the worst individual in the population
15:      end while
```



- meme options*



Inheriting Memetic Material – SIM

(1 0 0 0 1 + Hc1Hs1I2) $f=2$
 (1 1 0 1 1 + Hc0Hs4I1) $f=1$

(1 1 0 0 1 + Hc0Hs4I1)
 (1 0 0 1 1 + Hc0Hs4I1)

```
Individual_Level_Crossover(parent1, parent2)
BEGIN
  IF(both parents carry the same meme)
    Cross parents genetic material.
    Inherit common meme to offspring.
  ELSE-IF (parent1.fitness()==parent2.fitness())
    /* the two parents have different memes */
    /* but their fitness are comparable hence */
    /* a random choice is made */
    Cross parents genetic material.
    Choose a meme randomly from any of the two parents.
    Inherit selected meme to offspring.
  ELSE
    /* parents don't share memes nor fitness values */
    /* hence the fittest individual */
    /* imposes its meme preference */
    Cross parents genetic material.
    Choose meme from fittest parent.
    Inherit the chosen meme to offspring.
END
```

Krasnogor N, Smith JE (2001) Emergence of profitable search strategies based on a simple inheritance mechanism. In: Proc of the Genetic and Evolutionary Comp. Conf., pp 432–439
<http://eprints.uwe.ac.uk/11088/1/11088.pdf>



Mutating Memes during Evolution

- **Innovation (mutation) rate (IR)** $\in [0,1]$, is the probability of mutating the memes.
- Mutation randomly sets the meme option to one of the other options.
 - MAX-SAT: $(1\ 0\ 0\ 0\ 1\ +\ Hc2) \rightarrow 0: (1\ 0\ 0\ 0\ 1\ +\ Hc0)$
 - TSP: $(4\ 5\ 2\ 1\ 8\ 7\ 6\ 9\ 3\ +\ Mp2Mo1Cp1Co2W0Hc1Hs1I2) \rightarrow$
 $(4\ 5\ 2\ 1\ 8\ 7\ 6\ 9\ 3\ +\ Mp0Mo2Cp0Co1W1Hc3Hs0I4)$
- IR=0; no innovation, if a meme option is not introduced in the initial generation, it will not be reintroduced again.
- IR=1; All different strategies implied by the available M memes might be equally used.



Conclusion

- In general, a memetic algorithm performs better than a genetic algorithm.
- Choice of **meme (operators and their settings)** along with the encoding influence the performance of a memetic algorithm.
- Multimeme Algorithm can indeed *learn how to choose an operator and relevant settings through the evolutionary process (co-evolution)*.
 - There is a *trade-off* as learning requires (computation) time and a memetic algorithm with a single setting could perform better.



University of
Nottingham

UK | CHINA | MALAYSIA

Summary

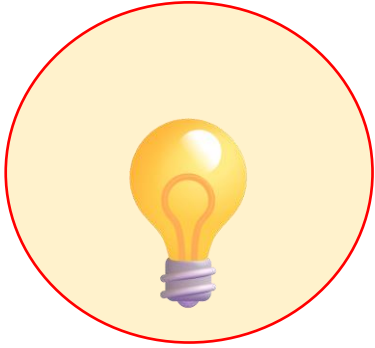


Overall Summary

- If there is *limited number of operators and settings*, MMA can be used.
- If the number of operators and settings are large, then additional analyses and methods could be needed to reduce the number of options leading to an MMA with improved performance.
- There is no single recipe for even choosing the set of memes while designing a memetic algorithm for solving a given problem.
- *Synergy* between memes/meme components could be possible.



Summary



1. Travelling Salesman Problem
2. Generic Permutation based Genetic Operators
3. Multimeme Memetic Algorithms
4. Benchmark Function Optimisation





University of
Nottingham

UK | CHINA | MALAYSIA

Questions



University of
Nottingham

UK | CHINA | MALAYSIA

NEXT:

Hyper-heuristics I